

Laporan Proyek Akhir
VISUALISASI GRAFIK FUNGSI KALKULUS 2D DAN 3D

Kalkulus II CR 001

Dosen Pengampu
Ir. NIXON ERZED , MT.



DISUSUN OLEH :

20240801042 Ramsay Abelson

20240801098 Wilson Fabian

Universitas Esa Unggul Tangerang

2026

Daftar Isi

BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Proyek	1
1.3 Ruang Lingkup	1
BAB II PROGRAM GRAFIK KALKULUS 2D	2
2.1 Pendahuluan Program 2D	2
2.2 Rancangan Sistem Program 2D	2
2.2.1 Deskripsi Umum	2
2.2.2 Blok Diagram	2
2.3 Format Masukan Data	4
2.4 Variable yang digunakan	4
2.5 Penjelasan Fungsi/Prosedur	4
2.6 Implementasi Program 2D	5
2.6.1 Kode Program Grafik 2D	5
2.6.2 Potongan Kode dan Penjelasan Setiap Fungsi	8
2.7 Hasil dan Pembahasan Program Grafik 2D	9
2.7.1 Hasil Tampilan Grafik 2D	9
2.8 Analisis Hasil	11
BAB III PROGRAM GRAFIK KALKULUS 3D	12
3.1 Pendahuluan Program 3D	12
3.2 Rancangan Sistem Program 3D	12
3.2.1 Deskripsi Umum	12
3.2.2 Blok Diagram	12
3.2.3 Flowchart Program 3D	13
3.3 Format Masukan Data	14
3.4 Variabel yang Digunakan	14
3.4 Implementasi Program 3D	14
3.4.1 Kode Program Grafik 3D	14
3.4.2 Potongan Kode dan Penjelasan Setiap Fungsi	17
3.5 Hasil dan Pembahasan Program 3D	19
3.6 Analisis Hasil	20
BAB IV KESIMPULAN	22

BAB I PENDAHULUAN

1.1 Latar Belakang

Kalkulus merupakan salah satu cabang matematika yang mempelajari perubahan serta hubungan antar variabel. Pada mata kuliah Kalkulus II, mahasiswa mempelajari fungsi dua variabel beserta representasinya dalam bentuk grafik dua dimensi (2D) dan tiga dimensi (3D). Konsep-konsep tersebut memiliki peran penting dalam berbagai bidang, seperti sains, teknik, dan teknologi.

Namun, pemahaman materi kalkulus sering kali menjadi tantangan apabila hanya disampaikan melalui pendekatan teoritis dan perhitungan manual. Banyak mahasiswa mengalami kesulitan dalam membayangkan bentuk grafik serta hubungan antar variabel yang direpresentasikan dalam fungsi matematika. Hal ini dapat berdampak pada kurang optimalnya pemahaman konsep dasar yang dipelajari.

Oleh karena itu, diperlukan media visual yang mampu menampilkan bentuk dan perilaku fungsi matematika secara lebih nyata dan interaktif. Melalui visualisasi grafik berbasis komputer, konsep kalkulus dapat dipahami dengan lebih mudah karena mahasiswa dapat melihat langsung perubahan grafik terhadap nilai variabel.

1.2 Tujuan Proyek

Tujuan dari pembuatan proyek ini adalah:

- Membuat program visualisasi grafik fungsi kalkulus dua dimensi (2D).
- Membuat program visualisasi grafik fungsi kalkulus tiga dimensi (3D).
- Membantu pemahaman konsep fungsi matematika melalui tampilan visual yang interaktif.
- Menerapkan konsep grafika komputer pada pembelajaran kalkulus menggunakan bahasa pemrograman C++.

1.3 Ruang Lingkup

Ruang lingkup proyek ini meliputi:

- Visualisasi beberapa fungsi matematika tertentu dalam bentuk grafik 2D dan 3D.
- Program berbasis desktop yang dijalankan pada komputer.
- Interaksi pengguna melalui perangkat input seperti keyboard dan mouse.
- Penggunaan bahasa pemrograman C++ dengan dukungan library grafika untuk menampilkan visualisasi grafik.

BAB II PROGRAM GRAFIK KALKULUS 2D

2.1 Pendahuluan Program 2D

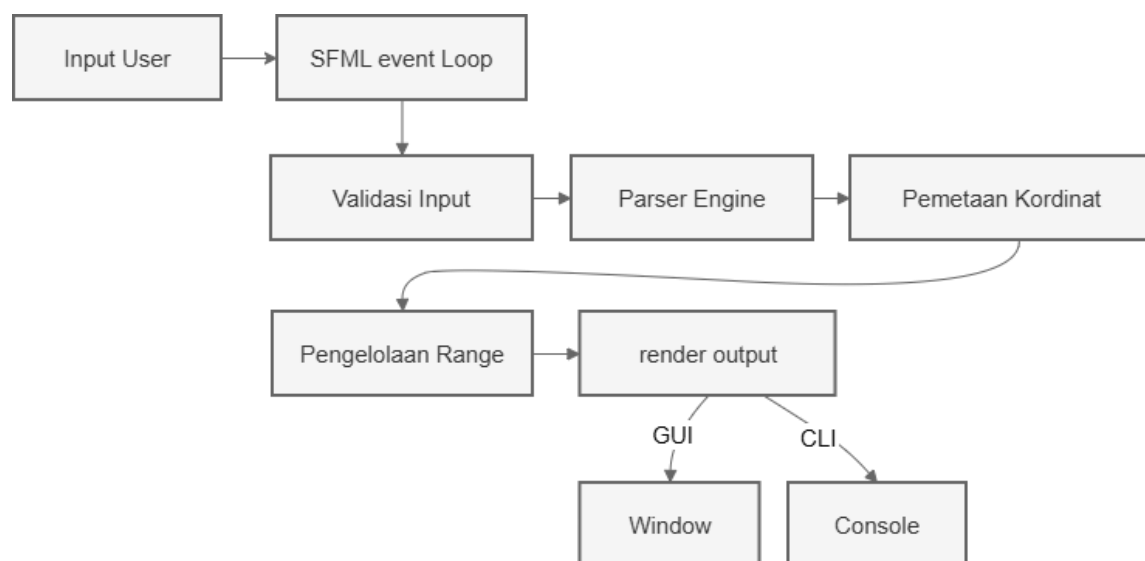
Program grafik kalkulus 2D merupakan aplikasi yang digunakan untuk menampilkan grafik fungsi satu variabel dalam bidang kartesius. Program ini memungkinkan pengguna untuk melihat bentuk kurva fungsi secara langsung berdasarkan fungsi dan rentang nilai tertentu.

2.2 Rancangan Sistem Program 2D

2.2.1 Deskripsi Umum

Program grafik 2D dibuat menggunakan bahasa pemrograman C++ dengan library SFML. Program mampu menerima input fungsi matematika dan rentang nilai x, kemudian menggambarkan grafik fungsi tersebut secara real-time.

2.2.2 Blok Diagram

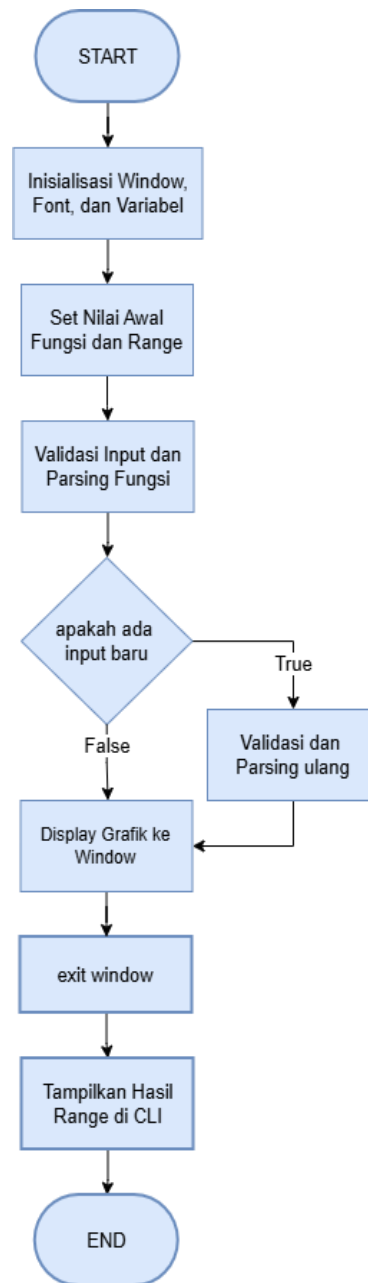


Gambar 1. Blok Diagram Program 2D

Blok diagram ini menunjukkan alur proses pembuatan visualisasi grafik dua dimensi (2D). Input dari pengguna diproses melalui SFML Event Loop, lalu divalidasi agar sesuai aturan. Setelah itu, Parser Engine mengolah fungsi matematika, kemudian dilakukan pemetaan koordinat dari nilai matematika ke posisi layar 2D.

Selanjutnya, pengelolaan range menentukan batas nilai sumbu X dan Y agar grafik terlihat proporsional. Hasil akhirnya dirender melalui modul render output, yang menampilkan grafik 2D pada window (GUI) dan console (CLI).

2.2.3 Flowchart Program 2D



Gambar 2. Flowchart Program 2D

Flowchart program 2D ini dimulai dari proses inisialisasi window, font, dan variabel yang diperlukan untuk menampilkan grafik. Setelah itu, program menetapkan nilai awal fungsi dan rentang (range) yang akan divisualisasikan, lalu melakukan validasi input serta parsing fungsi matematika.

Program kemudian memeriksa apakah terdapat input baru dari pengguna. Jika ada, proses validasi dan parsing akan diulang agar grafik selalu sesuai dengan input terbaru. Jika tidak ada input baru, grafik ditampilkan ke dalam window. Setelah window ditutup, program menampilkan hasil range pada CLI sebagai informasi tambahan, kemudian proses diakhiri.

2.3 Format Masukan Data

Program menerima input melalui GUI berupa teks yang diketik langsung menggunakan keyboard

- $f(x)$: string ekspresi matematika satu variabel x (contoh: $\sin(x)$, x^2+1).
- X_{Min} : nilai batas bawah domain x
- X_{Max} : nilai batas atas domain x
- Menekan Enter akan memperbarui rentang grafik.

2.4 Variable yang digunakan

Variabel berikut digunakan untuk menyimpan data, mengontrol alur program, serta mendukung proses perhitungan dan visualisasi grafik.

- **String**
strFunc : menyimpan fungsi matematika $f(x)$
strXmin, strXmax : menyimpan input batas domain x
- **Double**
xMin, xMax : batas domain sumbu X
yMin, yMax : batas domain sumbu Y
range, step, dx : parameter perhitungan skala dan plotting
- **Integer**
W, H : ukuran window grafik
activeField : penanda field input yang aktif
- **Struct Parser**
src : string ekspresi fungsi
pos : posisi parsing
Digunakan untuk mem-parsing dan mengevaluasi ekspresi matematika
- **SFML Object**
sf::RenderWindow win : window utama
sf::Font font : font teks
sf::FloatRect boxFunc, boxMin, boxMax : area input GUI

2.5 Penjelasan Fungsi/Prosedur

Program menerapkan beberapa fungsi dan prosedur untuk melakukan pemrosesan ekspresi matematika, konversi koordinat, serta visualisasi grafik fungsi.

- **Parser**
Memproses dan menghitung nilai fungsi $f(x)$ dari ekspresi matematika.
- **sx dan sy**
Mengonversi koordinat matematika ke koordinat piksel layar.
- **fnt**
Memformat angka agar tampil ringkas pada sumbu grafik.
- **updateRange**
Memperbarui batas sumbu X dan Y berdasarkan input pengguna.

2.6 Implementasi Program 2D

2.6.1 Kode Program Grafik 2D

```
#include <SFML/Graphics.hpp>
#include <cmath>
#include <string>
#include <vector>
#include <sstream>
#include <iomanip>
#include <iostream>

using namespace std;
struct Parser {
    string src; size_t pos;
    double parseExpr(double x) {
        double res = parseTerm(x);
        while (pos < src.size() && (src[pos] == '+' || src[pos] == '-')) {
            char op = src[pos++]; double rhs = parseTerm(x);
            if (op == '+') res += rhs; else res -= rhs;
        }
        return res;
    }
    double parseTerm(double x) {
        double res = parseFactor(x);
        while (pos < src.size() && (src[pos] == '*' || src[pos] == '/')) {
            char op = src[pos++]; double rhs = parseFactor(x);
            if (op == '*') res *= rhs; else res /= rhs;
        }
        return res;
    }
    double parseFactor(double x) {
        double res = parseBase(x);
        if (pos < src.size() && src[pos] == '^') { pos++; res = pow(res, parseFactor(x)); }
        return res;
    }
    double parseBase(double x) {
        while(pos < src.size() && src[pos] == ' ') pos++;
        if (pos >= src.size()) return 0;
        if (isdigit(src[pos]) || src[pos] == '.') {
            size_t end; double v = stod(src.substr(pos), &end); pos += end; return v;
        }
        if (isalpha(src[pos])) {
            string f; while(pos < src.size() && isalpha(src[pos])) f += src[pos++];
            if (f == "x") return x;
            if (src[pos] == '(') {
                pos++; double arg = parseExpr(x); if (src[pos] == ')') pos++;
                if (f == "sin") return sin(arg); if (f == "cos") return cos(arg); if (f == "tan") return tan(arg);
                if (f == "sqrt") return sqrt(arg); if (f == "log") return log(arg); if (f == "exp") return exp(arg);
                if (f == "abs") return abs(arg);
            }
        }
        if (src[pos] == '(') { pos++; double v = parseExpr(x); if (src[pos] == ')') pos++; return v; }
        if (src[pos] == '-') { pos++; return -parseBase(x); }
        return 0;
    }
    double solve(string s, double x) { src = s; pos = 0; try { return parseExpr(x); } catch(...) { return 0; } }
} parser;

double xMin = -10.0, xMax = 10.0;
double yMin = -6.0, yMax = 6.0;
const int W = 1200, H = 700;

float sx(double x) { return (float)((x - xMin) / (xMax - xMin) * W); }
float sy(double y) { return (float)(H - (y - yMin) / (yMax - yMin) * H); }
```

```

string fmt(double v) {
    stringstream ss; ss << fixed << setprecision(2) << v;
    string s = ss.str();
    if(s.find('.')!=string::npos){ while(s.back()=='0') s.pop_back(); if(s.back()!='.') s.pop_back(); }
    return s;
}

int main() {
    sf::RenderWindow win(sf::VideoMode(W, H + 100), "Graph Calc - Tutup Window untuk Lihat Hasil CLI");
    win.setFramerateLimit(60);
    sf::Font font;

    if (!font.loadFromFile("C:\\Users\\ramsa\\Coding\\Kuliah\\Calculus_2D\\resources\\OpenSans-Regular.ttf")) return 1;
    if (!font.loadFromFile("OpenSans-Regular.ttf")) {

        cout << "Font gagal dimuat!" << endl;

        return 1;
    }

    string strFunc = "sin(x)";
    string strXmin = "-10";
    string strXmax = "10";

    int activeField = 0;
    sf::FloatRect boxFunc(80, H+20, 400, 30);
    sf::FloatRect boxMin(600, H+20, 100, 30);
    sf::FloatRect boxMax(800, H+20, 100, 30);
    auto updateRange = [&]() {
        try {
            double newMin = stod(strXmin); double newMax = stod(strXmax);
            if (newMin < newMax) { xMin = newMin; xMax = newMax; }
            double rangeY = (xMax - xMin) * ((double)H/W);
            yMin = -(rangeY/2.0); yMax = (rangeY/2.0);
        } catch (...) {}
    };
    updateRange();

    while (win.isOpen()) {
        sf::Event ev;
        while (win.pollEvent(ev)) {
            if (ev.type == sf::Event::Closed) win.close();
            if (ev.type == sf::Event::MouseButtonPressed) {
                sf::Vector2f m((float)ev.mouseButton.x, (float)ev.mouseButton.y);
                if (boxFunc.contains(m)) activeField = 0;
                else if (boxMin.contains(m)) activeField = 1;
                else if (boxMax.contains(m)) activeField = 2;
            }
            if (ev.type == sf::Event::TextEntered) {
                if (ev.text.unicode < 128) {
                    char c = (char)ev.text.unicode;
                    string* target = (activeField==0 ? &strFunc : (activeField==1 ? &strXmin : &strXmax));
                    if (c == 8) { if (!target->empty()) target->pop_back(); }
                    else if (c == 13) { updateRange(); }
                    else if (c >= 32) { *target += c; }
                }
            }
        }
        win.clear(sf::Color::White);

        double range = xMax - xMin;
        double step = pow(10, floor(log10(range))) / 2.0;
        if (range / step > 20) step *= 2;
    }
}

```



```

for (double x = floor(xMin/step)*step; x <= xMax; x += step) {
    float px = sx(x);
    if(px >= 0 && px <= W) {
        win.draw(new sf::Vertex[2]{{px, 0}, sf::Color(220,220,220)}, {{px, (float)H}, sf::Color(220,220,220)}, 2, sf::Lines);
        sf::Text t(fmt(x), font, 12); t.setFillColors(sf::Color::Black); t.setPosition(px+2, sy(0)+2); win.draw(t);
    }
}
for (double y = floor(yMin/step)*step; y <= yMax; y += step) {
    float py = sy(y);
    if(py >= 0 && py <= H) {
        win.draw(new sf::Vertex[2]{{0, py}, sf::Color(220,220,220)}, {{(float)W, py}, sf::Color(220,220,220)}, 2, sf::Lines);
        sf::Text t(fmt(y), font, 12); t.setFillColors(sf::Color::Black); t.setPosition(sx(0)+2, py-16); win.draw(t);
    }
}

sf::RectangleShape axX(sf::Vector2f(W, 2)); axX.setPosition(0, sy(0)); axX.setFillColors(sf::Color::Black); win.draw(axX);
sf::RectangleShape axY(sf::Vector2f(2, H)); axY.setPosition(sx(0), 0); axY.setFillColors(sf::Color::Black); win.draw(axY);
sf::VertexArray graph(sf::LineStrip);
double dx = range / W;
for (int i = 0; i <= W; ++i) {
    double x = xMin + i * dx;
    double y = parser.solve(strFunc, x);
    if (std::isfinite(y)) {
        float py = sy(y);
        if (py > -H && py < 2*H) graph.append(sf::Vertex(sf::Vector2f((float)i, py), sf::Color::Blue));
    } else if (graph.getVertexCount() > 0) { win.draw(graph); graph.clear(); }
}
win.draw(graph);

sf::RectangleShape uiBg(sf::Vector2f(W, 100)); uiBg.setPosition(0, H); uiBg.setFillColors(sf::Color(240,240,240));
uiBg.setOutlineThickness(1); uiBg.setOutlineColors(sf::Color::Black); win.draw(uiBg);

auto drawBox = [&](sf::FloatRect r, string s, string l, int id) {
    sf::RectangleShape box(sf::Vector2f(r.width, r.height));
    box.setPosition(r.left, r.top); box.setFillColors(sf::Color::White);
    box.setOutlineThickness(activeField == id ? 2 : 1); box.setOutlineColors(activeField == id ? sf::Color::Blue : sf::Color(100,100,100));
    win.draw(box);
    sf::Text t(s, font, 18); t.setFillColors(sf::Color::Black); t.setPosition(r.left+5, r.top+2); win.draw(t);
    sf::Text lb(l, font, 14); lb.setFillColors(sf::Color::Black); lb.setPosition(r.left, r.top-20); win.draw(lb);
};
drawBox(boxFunc, strFunc, "f(x) =", 0);
drawBox(boxMin, strXmin, "X Min", 1);
drawBox(boxMax, strXmax, "X Max", 2);
win.display();
}

cout << "\n\n";
cout << "===== " << endl;
cout << "    HASIL PERHITUNGAN AKHIR (CLI)    " << endl;
cout << "===== " << endl;
cout << " Fungsi : " << strFunc << endl;
cout << " Range : " << xMin << " s/d " << xMax << endl;
cout << "-----" << endl;
cout << "    x    |    f(x)    " << endl;
cout << "-----" << endl;

double stepCli = (xMax - xMin) / 20.0;
if (stepCli <= 0.0001) stepCli = 0.1;
for (double x = xMin; x <= xMax + (stepCli/10.0); x += stepCli) {
    double y = parser.solve(strFunc, x);
    cout << fixed << setprecision(3) << setw(10) << x << "    | ";
    if (isfinite(y)) cout << setw(12) << y << endl;
    else cout << "    NaN    " << endl;
}
cout << "-----" << endl; cout << "Tekan Enter untuk keluar..."; cin.get();
return 0;
}

```

2.6.2 Potongan Kode dan Penjelasan Setiap Fungsi

Struct Parser

Struct Parser digunakan untuk membaca dan menghitung ekspresi matematika yang dimasukkan pengguna dalam bentuk string, misalnya $\sin(x)$ atau $x^2 + 3 \cdot x$.

double parseExpr(double x)

Mengolah ekspresi matematika tingkat atas yang mengandung operator penjumlahan (+) dan pengurangan (-). Fungsi ini menjadi level tertinggi dalam proses parsing ekspresi.

double parseTerm(double x)

Mengolah operasi perkalian (*) dan pembagian (/). Fungsi ini memiliki prioritas lebih tinggi dibanding penjumlahan dan pengurangan.

double parseFactor(double x)

Mengolah operasi perpangkatan (^). Digunakan untuk memastikan urutan operasi matematika (prioritas operator) berjalan dengan benar.

double parseBase(double x)

Mengenali dan memproses:

- Bilangan numerik
- Variabel x
- Fungsi matematika seperti sin, cos, tan, log, sqrt, dan exp
- Ekspresi dalam tanda kurung

double solve(string s, double x)

Fungsi utama untuk evaluasi matematika. Mengambil ekspresi fungsi dalam bentuk string dan nilai x, lalu mengembalikan hasil perhitungan f(x).

Fungsi Konversi Koordinat

- **float sx(double x)**

Mengonversi nilai koordinat matematis sumbu X menjadi posisi piksel pada layar.

- **float sy(double y)**

Mengonversi nilai koordinat matematis sumbu Y menjadi posisi piksel pada layar dengan sistem koordinat grafika.

Fungsi Pendukung Tampilan

- **string fmt(double v)**

Memformat angka ke dalam bentuk string dengan presisi tertentu agar label sumbu terlihat rapi dan mudah dibaca.

- **updateRange (lambda function)**

Menghitung ulang nilai **xMin**, **xMax**, **yMin**, dan **yMax** berdasarkan input pengguna sehingga grafik dapat menyesuaikan skala tampilan secara otomatis.

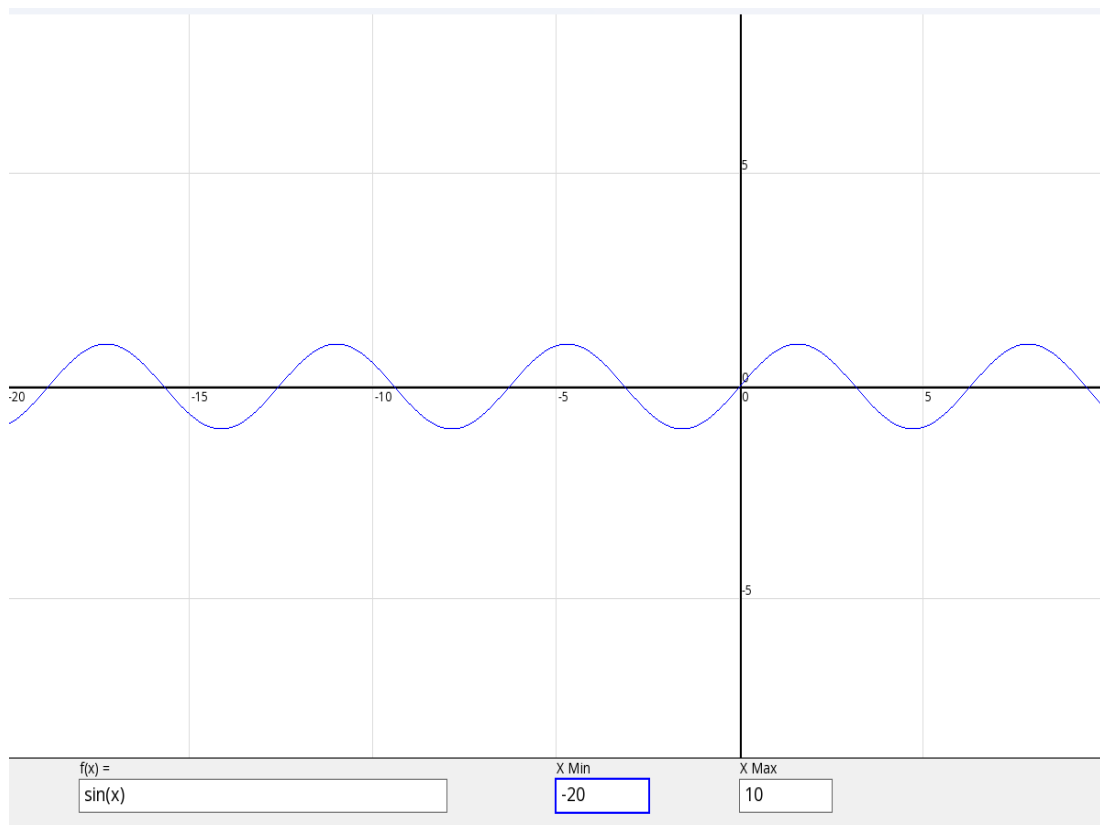
2.7 Hasil dan Pembahasan Program Grafik 2D

2.7.1 Hasil Tampilan Grafik 2D

Program menampilkan grafik fungsi satu variabel dalam sistem koordinat Kartesius dua dimensi. Pengguna dapat memasukkan fungsi matematika, serta menentukan rentang nilai sumbu X.

Contoh input:

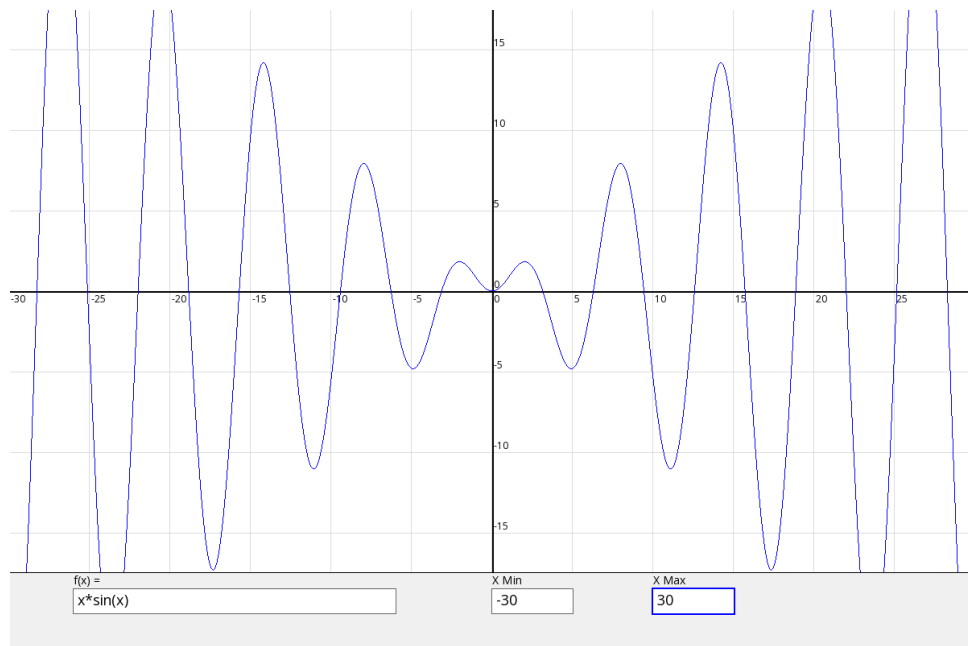
- Fungsi: **sin(x)**
- Rentang X: **-20** sampai **10**



Gambar 3. Visualisasi Grafik Fungsi sin(x)

Contoh input:

- Fungsi: $x*\sin(x)$
- Rentang X: -30 sampai 30



Gambar 4. Visualisasi Grafik Fungsi $x*\sin(x)$

Setelah jendela ditutup, hasil perhitungan numerik fungsi akan ditampilkan pada terminal (CLI).

```

Fungsi : sin(x)
Range : -20 s/d 10
-----

```

x	f(x)
-20.000	-0.913
-18.500	0.342
-17.000	0.961
-15.500	-0.206
-14.000	-0.991
-12.500	0.066
-11.000	1.000
-9.500	0.075
-8.000	-0.989
-6.500	-0.215
-5.000	0.959
-3.500	0.351
-2.000	-0.909
-0.500	-0.479
1.000	0.841
2.500	0.598
4.000	-0.757
5.500	-0.706
7.000	0.657
8.500	0.798
10.000	-0.544

Gambar 5. Output Perhitungan $\sin(x)$ pada Terminal

```

Fungsi : x*sin(x)
Range : -30 s/d 30
-----

```

x	f(x)
-30.000	-29.641
-27.000	25.822
-24.000	-21.734
-21.000	17.570
-18.000	-13.518
-15.000	9.754
-12.000	-6.439
-9.000	3.709
-6.000	-1.676
-3.000	0.423
0.000	0.000
3.000	0.423
6.000	-1.676
9.000	3.709
12.000	-6.439
15.000	9.754
18.000	-13.518
21.000	17.570
24.000	-21.734
27.000	25.822
30.000	-29.641

Gambar 6. Output Perhitungan $x*\sin(x)$ pada Terminal

2.8 Analisis Hasil

Penjelasan Program Grafik Kalkulus 2D

Program grafik kalkulus 2D ini dikembangkan menggunakan bahasa pemrograman C++ dengan tujuan memvisualisasikan fungsi matematika satu variabel dalam bentuk grafik dua dimensi. Visualisasi ini membantu pengguna memahami karakteristik fungsi secara lebih nyata dibandingkan perhitungan simbolik atau tabel nilai semata. Melalui tampilan grafik, perubahan nilai fungsi terhadap sumbu X dapat diamati secara langsung.

Fungsi Matematika yang Ditampilkan

Program menampilkan grafik fungsi dengan satu variabel bebas, yaitu X, dan satu variabel terikat, yaitu Y. Nilai Y diperoleh dari hasil perhitungan fungsi matematika yang telah ditentukan atau diinput oleh pengguna. Setiap perubahan fungsi akan menghasilkan bentuk grafik yang berbeda sesuai dengan sifat matematis fungsi tersebut, seperti fungsi periodik, fungsi polinomial, maupun fungsi logaritmik.

Pengaruh Rentang dan Skala Nilai X

Rentang nilai X memiliki peran penting dalam menentukan tampilan grafik yang dihasilkan. Perubahan batas minimum dan maksimum X secara langsung memengaruhi:

- Bentuk grafik yang terlihat pada layar
- Skala tampilan grafik
- Kerapatan titik-titik pembentuk grafik

Rentang yang terlalu sempit dapat menyebabkan grafik terlihat terpotong, sedangkan rentang yang terlalu lebar dapat membuat detail kurva menjadi kurang jelas.

Kerapatan Titik dan Interval Perhitungan

Grafik dibentuk dari sekumpulan titik hasil perhitungan nilai fungsi pada interval X tertentu. Semakin kecil interval pengambilan nilai X, semakin banyak titik yang dihitung dan digambar pada layar. Hal ini menyebabkan grafik yang dihasilkan terlihat lebih halus dan mendekati bentuk kurva sebenarnya. Sebaliknya, interval yang terlalu besar dapat membuat grafik tampak patah-patah dan kurang merepresentasikan bentuk fungsi secara akurat.

Manfaat Visualisasi Grafik 2D

Dengan adanya visualisasi grafik dua dimensi, pengguna dapat lebih mudah:

- Memahami hubungan antara nilai X dan Y
- Mengamati perubahan perilaku fungsi akibat perubahan parameter atau rentang

BAB III PROGRAM GRAFIK KALKULUS 3D

3.1 Pendahuluan Program 3D

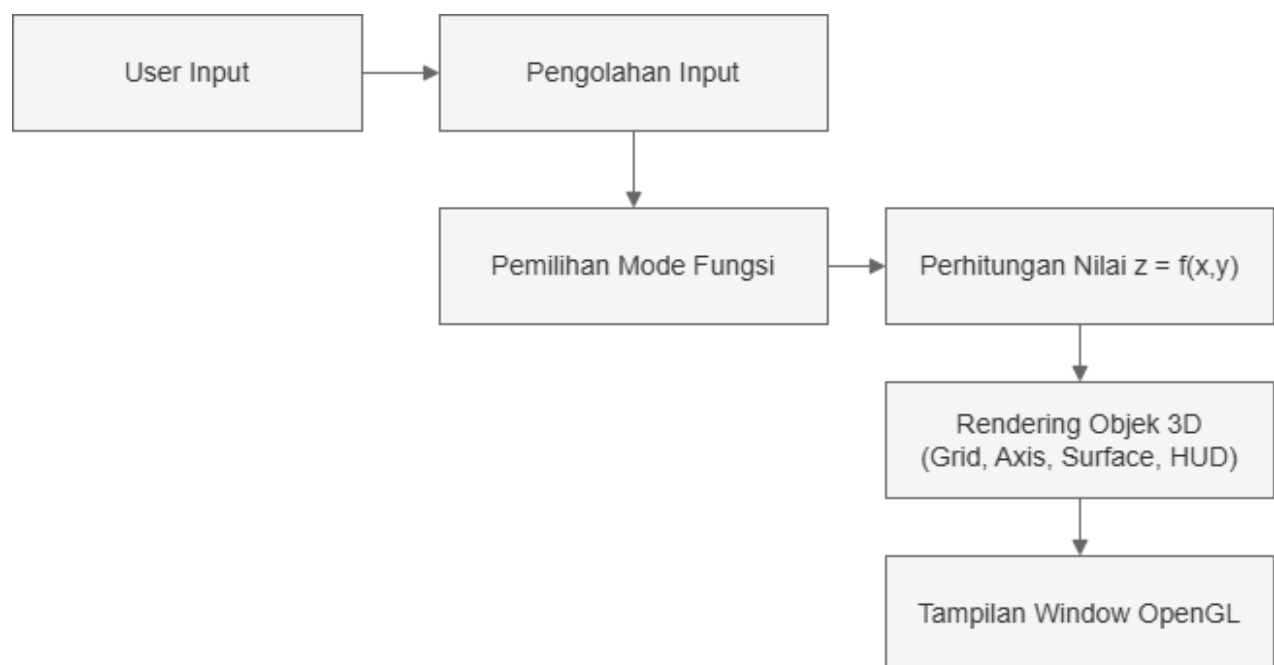
Program grafik kalkulus 3D merupakan aplikasi yang digunakan untuk memvisualisasikan fungsi dua variabel dalam ruang tiga dimensi. Program ini menampilkan hubungan antara variabel x , y , dan z dalam bentuk permukaan grafik sehingga mempermudah pemahaman konsep fungsi multivariabel pada mata kuliah Kalkulus II.

3.2 Rancangan Sistem Program 3D

3.2.1 Deskripsi Umum

Program grafik 3D dibuat menggunakan bahasa pemrograman C++ dengan bantuan library OpenGL dan GLUT. Program ini mampu menampilkan grafik permukaan fungsi dua variabel serta menyediakan fitur interaksi berupa rotasi dan zoom menggunakan mouse dan keyboard.

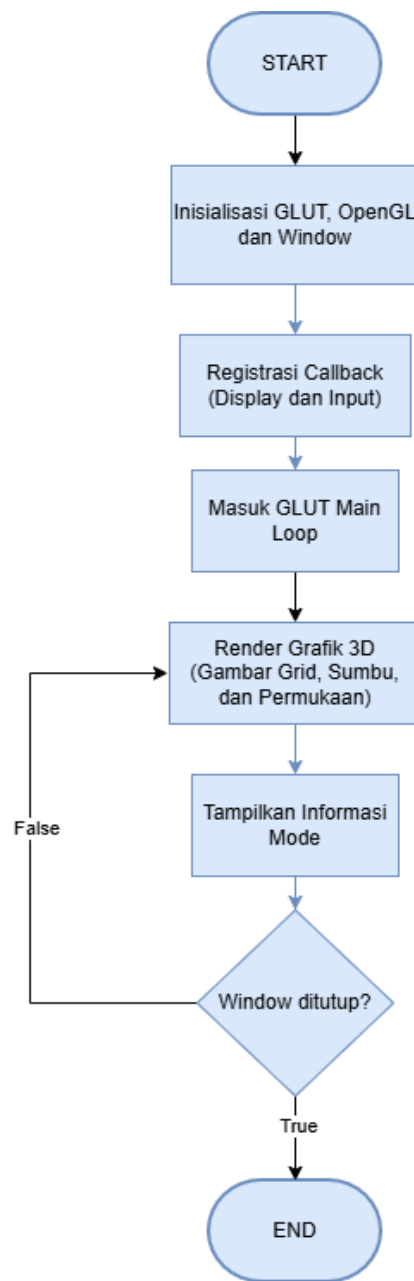
3.2.2 Blok Diagram



Gambar 7. Blok Diagram Program 3D

Blok diagram program 3D menunjukkan alur pemrosesan data dari input pengguna hingga visualisasi grafik tiga dimensi. Proses diawali dari *user input* yang kemudian diolah untuk menentukan mode fungsi yang digunakan. Berdasarkan mode tersebut, sistem melakukan perhitungan nilai $z = f(x, y)$. Hasil perhitungan ini selanjutnya digunakan pada tahap rendering objek 3D, yang mencakup penggambaran grid, sumbu koordinat, permukaan grafik, serta elemen informasi (HUD), lalu ditampilkan pada window berbasis OpenGL.

3.2.3 Flowchart Program 3D



Gambar 8. Flowchart Program 3D

Flowchart program 3D menjelaskan urutan eksekusi program secara runtime. Program dimulai dengan inisialisasi GLUT, OpenGL, dan window, kemudian dilanjutkan dengan registrasi callback untuk tampilan dan input.

Setelah masuk ke GLUT main loop, sistem secara berulang merender grafik 3D dan menampilkan informasi mode selama window masih aktif. Proses ini akan terus berjalan sampai pengguna menutup window, yang menandai akhir dari eksekusi program.

3.3 Format Masukan Data

Masukan data pada program grafik 3D dilakukan melalui:

Keyboard

- Tombol **1** untuk menampilkan fungsi pertama
- Tombol **2** untuk menampilkan fungsi kedua
- Tombol **3** untuk menampilkan fungsi ketiga

Mouse

- Klik kiri dan geser untuk rotasi grafik
- Scroll mouse untuk memperbesar dan memperkecil tampilan (zoom)

3.4 Variabel yang Digunakan

Variabel berikut digunakan untuk mengatur tampilan, interaksi, dan perhitungan grafik 3D.

Float

- **rotX, rotY** : sudut rotasi kamera
- **zoom** : jarak kamera terhadap objek
- **x, y, z** : variabel koordinat grafik

Integer

- **lastX, lastY** : posisi terakhir mouse
- **mode** : penanda fungsi matematika yang aktif

Boolean

- **mouseDown** : status tombol mouse

3.4 Implementasi Program 3D

3.4.1 Kode Program Grafik 3D

```
#include <GL/glut.h>
#include <cmath>
#include <string>
#include <cstdio>

float rotX = 30, rotY = -45;
float zoom = 15;
int lastX, lastY;
bool mouseDown = false;
int mode = 1;
float fungsi(float x, float y) {
    if (mode == 1) return sin(x) * cos(y);
    if (mode == 2) return x*x - y*y;
    if (mode == 3) return log(x*x + y*y + 1);
    return 0;
}
std::string num(float v) {
    char buf[16];
    sprintf(buf, "%.0f", v);
    return buf;
```



```

}

void drawText(float x, float y, float z, const std::string& t) {
    glRasterPos3f(x, y, z);
    for (char c : t)
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, c);
}

void drawGrid() {
    glColor3f(0.85,0.85,0.85);
    glBegin(GL_LINES);
    for (int i=-5;i<=5;i++) {
        glVertex3f(i,-5,0); glVertex3f(i,5,0);
        glVertex3f(-5,i,0); glVertex3f(5,i,0);
    } glEnd();
}

void drawBox() {
    glColor3f(0.6,0.6,0.6);
    glBegin(GL_LINES);
    float s = 5;
    float p[8][3] = {
        {-s,-s,-s},{ s,-s,-s},{ s, s,-s},{-s, s,-s},
        {-s,-s, s},{ s,-s, s},{ s, s, s},{-s, s, s}
    };
    int e[12][2] = {
        {0,1},{1,2},{2,3},{3,0},
        {4,5},{5,6},{6,7},{7,4},
        {0,4},{1,5},{2,6},{3,7}
    };
    for (auto &i : e) {
        glVertex3fv(p[i[0]]);
        glVertex3fv(p[i[1]]);
    } glEnd();
}

void drawAxis() {
    glLineWidth(2);
    glBegin(GL_LINES);
    glColor3f(1,0,0); glVertex3f(-6,0,0); glVertex3f(6,0,0);
    glColor3f(0,1,0); glVertex3f(0,-6,0); glVertex3f(0,6,0);
    glColor3f(0,0,1); glVertex3f(0,0,-6); glVertex3f(0,0,6);
    glEnd();
    glColor3f(0,0,0);
    drawText(6.2,0,0,"X");
    drawText(0,6.2,0,"Y");
    drawText(0,0,6.2,"Z");
    for (int i=-5;i<=5;i++) {
        if (i==0) continue;
        drawText(i, -0.3, 0, num(i));
        drawText(-0.4, i, 0, num(i));
        drawText(0, -0.3, i, num(i));
    }
}

void drawZLevels() {
    glColor3f(0.8,0.8,0.8);
    for (int z=-2; z<=2; z++) {
        glBegin(GL_LINE_LOOP);
        glVertex3f(-5,-5,z);
        glVertex3f( 5,-5,z);
        glVertex3f( 5, 5,z);
        glVertex3f(-5, 5,z);
        glEnd();
        drawText(5.1,5.1,z,num(z));
    }
}

void drawSurface() {
    glColor3f(0.6,0.1,0.7);
    for (float x=-2; x<=2; x+=0.1f) {
        glBegin(GL_LINE_STRIP);
        for (float y=-2; y<=2; y+=0.1f) {
            glVertex3f(x, y, fungsi(x,y));
        }
    }
}

```

```

        glEnd();
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glTranslatef(0,0,-zoom);
    glRotatef(rotX,1,0,0);
    glRotatef(rotY,0,1,0);

    drawGrid();
    drawBox();
    drawAxis();
    drawZLevels();
    drawSurface();
    glutSwapBuffers();
}

void reshape(int w,int h) {
    glViewport(0,0,w,h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45,(float)w/h,1,100);
    glMatrixMode(GL_MODELVIEW);
}

void mouse(int b,int s,int x,int y) {
    if (b == GLUT_LEFT_BUTTON) {
        mouseDown = (s == GLUT_DOWN);
        lastX = x; lastY = y;
    }
    if (b == 3) zoom -= 0.5f;
    if (b == 4) zoom += 0.5f;
    glutPostRedisplay();
}

void motion(int x,int y) {
    if (!mouseDown) return;
    rotY += (x - lastX);
    rotX += (y - lastY);
    lastX = x; lastY = y;
    glutPostRedisplay();
}

void keyboard(unsigned char k,int,int) {
    if (k=='1') mode=1;
    if (k=='2') mode=2;
    if (k=='3') mode=3;
    glutPostRedisplay();
}

void init() {
    glEnable(GL_DEPTH_TEST);
    glClearColor(1,1,1,1);
}

int main(int argc,char** argv) {
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(1000,800);
    glutCreateWindow("Grafik 3D Kalkulus");
    init();
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMouseFunc(mouse);
    glutMotionFunc(motion);
    glutKeyboardFunc(keyboard);
    glutMainLoop();
    return 0;
}

```

3.4.2 Potongan Kode dan Penjelasan Setiap Fungsi

float fungsi(float x, float y)

Menghitung nilai fungsi matematika berdasarkan mode yang sedang aktif. Mode 1 menampilkan fungsi trigonometri, mode 2 menampilkan fungsi aljabar kuadrat, dan mode 3 menampilkan fungsi logaritmik.

void drawGrid()

Menggambar garis kisi (grid) pada bidang XY sebagai acuan visual untuk melihat posisi grafik dalam ruang 3D.

void drawBox()

Menggambar kubus batas (bounding box) yang berfungsi sebagai pembatas area visualisasi grafik agar ruang 3D terlihat lebih jelas.

void drawAxis()

Menggambar sumbu X, Y, dan Z beserta label dan angka skala untuk menunjukkan sistem koordinat kartesius tiga dimensi.

void drawSurface()

Menggambar grafik permukaan 3D berdasarkan fungsi matematika yang dipilih. Permukaan dibuat menggunakan garis berulang dengan interval kecil agar grafik terlihat lebih halus.

void drawText(float x, float y, float z, string t)

Menampilkan teks dalam ruang 3D, digunakan untuk memberi label sumbu dan angka skala.

void drawText2D(float x, float y, string t)

Menampilkan teks dua dimensi pada layar sebagai bagian dari HUD (Heads-Up Display), seperti informasi mode fungsi.

void drawHUD(int w, int h)

Menampilkan informasi mode fungsi yang sedang aktif serta petunjuk penggunaan keyboard di layar tanpa terpengaruh rotasi kamera.

void display()

Fungsi utama untuk menampilkan seluruh objek grafika, mulai dari grid, sumbu, grafik permukaan, hingga HUD. Fungsi ini dipanggil setiap kali layar diperbarui.

void reshape(int w, int h)

Menyesuaikan tampilan grafika ketika ukuran jendela berubah agar perspektif 3D tetap proporsional.

void mouse(int b, int s, int x, int y)

Mengatur interaksi mouse, yaitu rotasi grafik menggunakan klik kiri dan pengaturan zoom menggunakan scroll mouse.

void motion(int x, int y)

Mengubah sudut pandang kamera saat mouse digerakkan sambil ditekan, sehingga grafik dapat diputar secara interaktif.

void keyboard(unsigned char k, int, int)

Mengatur input keyboard untuk mengganti mode fungsi grafik:

- Tombol 1 untuk fungsi pertama
- Tombol 2 untuk fungsi kedua
- Tombol 3 untuk fungsi ketiga

void init()

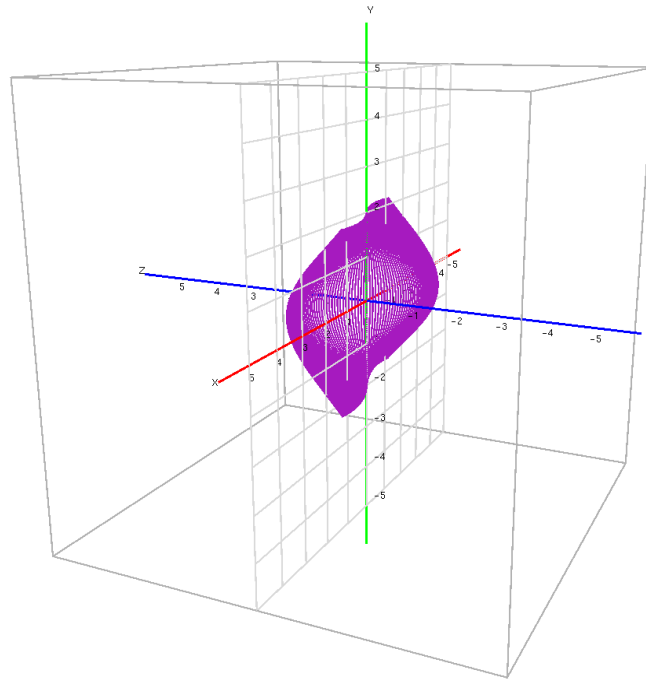
Melakukan inisialisasi awal OpenGL seperti mengaktifkan depth test, smoothing garis, dan mengatur warna latar belakang.

int main(int argc, char argv)**

Fungsi utama program yang bertugas menginisialisasi GLUT, membuat jendela grafika, mengatur callback fungsi, dan menjalankan loop utama aplikasi.

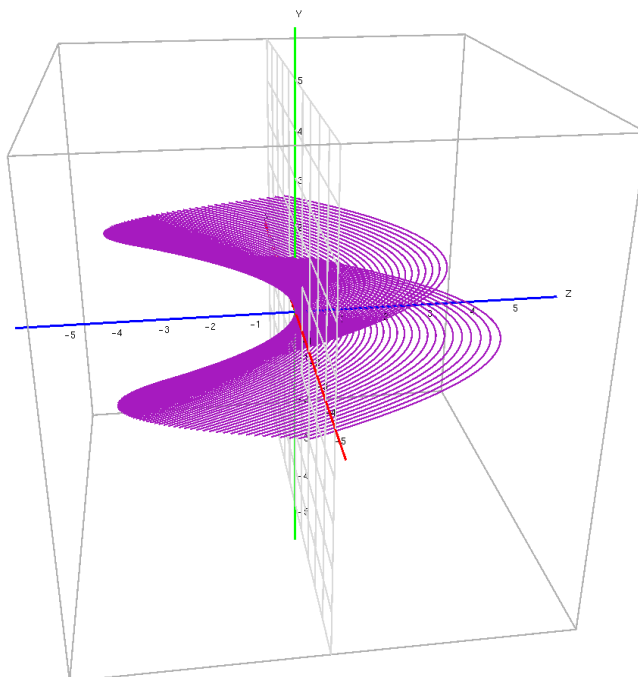
3.5 Hasil dan Pembahasan Program 3D

Fungsi 1 : $z = \sin(x) \cos(y)$



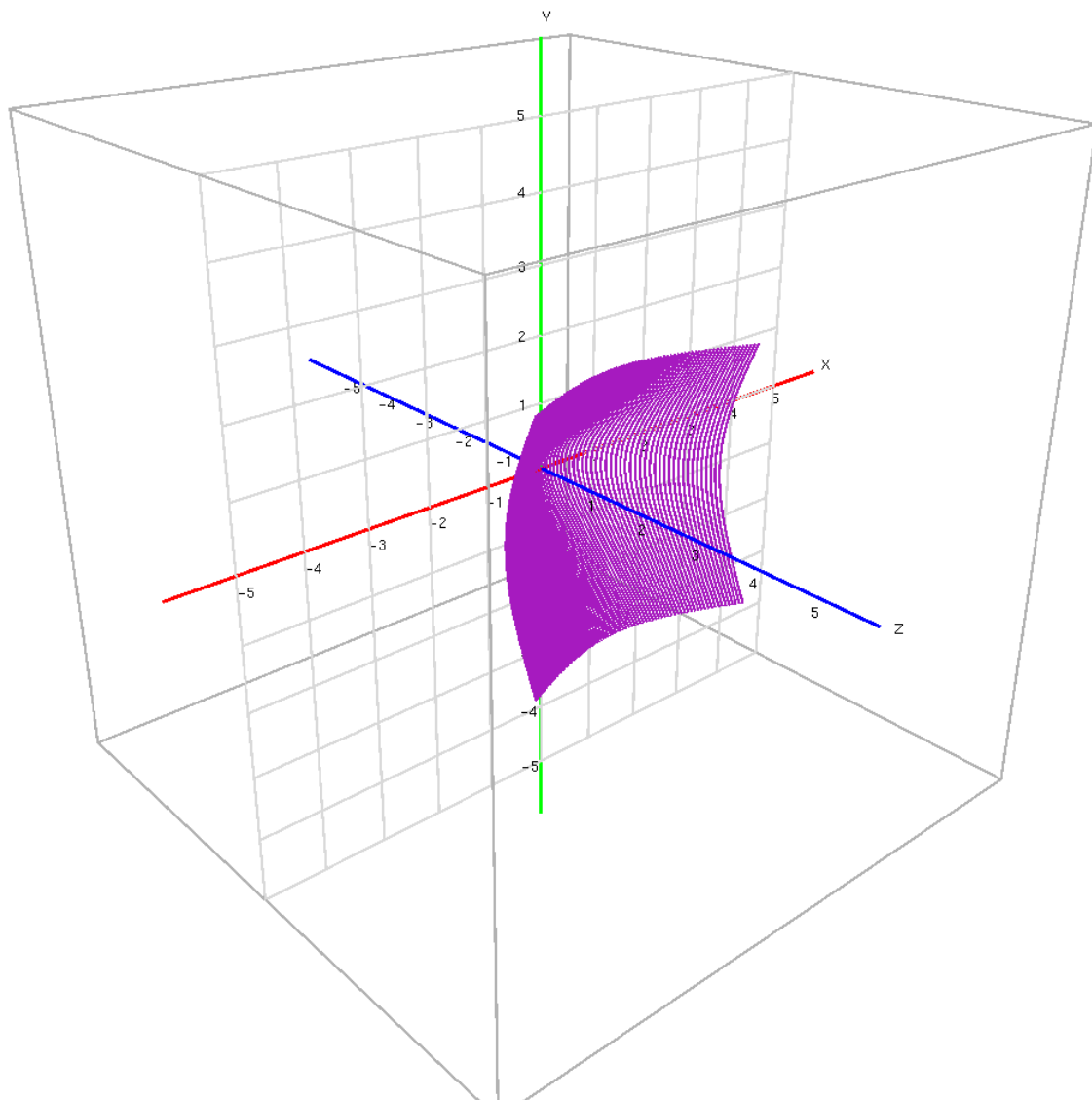
Gambar 9. Visualisasi Grafik 3D Fungsi $z = \sin(x) \cos(y)$

Fungsi 2 : $z = x^2 - y^2$



Gambar 10. Visualisasi Grafik 3D Fungsi $z = x^2 - y^2$

Fungsi 3: $\ln(x^2 + y^2 + 1)$



Gambar 11. Visualisasi Grafik 3D Fungsi $\ln(x^2 + y^2 + 1)$

3.6 Analisis Hasil

Penjelasan Program Grafik Kalkulus 3D

Program grafik kalkulus 3D ini dibuat menggunakan bahasa pemrograman **C++** dengan bantuan library **OpenGL** dan **GLUT**. Program berfungsi untuk memvisualisasikan grafik permukaan dari fungsi dua variabel dalam ruang tiga dimensi sehingga bentuk dan karakteristik fungsi dapat diamati secara lebih jelas.

Fungsi Matematika yang Ditampilkan

Program menampilkan grafik permukaan dengan variabel bebas x dan y , serta variabel terikat z . Nilai z dihitung melalui fungsi matematika yang dapat diganti oleh pengguna menggunakan input keyboard.

Fungsi yang disediakan pada program ini adalah:

- $Z = \sin(x) \cos(y)$
- $Z = x^2 - y^2$
- $Z = \ln(x^2 + y^2 + 1)$

Pergantian fungsi dilakukan dengan menekan tombol **1**, **2**, atau **3** pada keyboard.

Sistem Koordinat dan Visualisasi

Program menampilkan:

- **Sumbu X, Y, dan Z** sebagai acuan posisi grafik
- **Grid bidang XY** untuk mempermudah pembacaan posisi
- **Kubus pembatas** sebagai batas ruang visualisasi
- **Level Z** untuk membantu melihat ketinggian grafik terhadap sumbu Z

Label angka dan huruf pada setiap sumbu membantu pengguna membaca skala nilai dengan lebih mudah.

Proses Penggambaran Grafik Permukaan

Grafik permukaan dibuat dengan cara:

- Mengambil nilai x dan y dalam interval tertentu
- Menghitung nilai z menggunakan fungsi matematika
- Menghubungkan titik-titik tersebut menjadi garis sehingga membentuk permukaan grafik

Semakin kecil jarak antar titik, maka grafik yang dihasilkan akan terlihat lebih halus.

Interaksi Pengguna

Program bersifat interaktif dan menyediakan beberapa fitur pengendalian tampilan, yaitu:

- **Rotasi grafik** menggunakan mouse dengan menekan tombol kiri
- **Perbesaran dan pengecilan tampilan (zoom)** menggunakan scroll mouse
- **Pergantian fungsi grafik** melalui input keyboard

Fitur ini memudahkan pengguna mengamati grafik dari berbagai sudut pandang.

BAB IV KESIMPULAN

Kesimpulan

Berdasarkan proses perancangan, implementasi, serta pengujian yang telah dilakukan, dapat disimpulkan bahwa proyek akhir mata kuliah Kalkulus II berupa program visualisasi grafik kalkulus 2D dan 3D berhasil direalisasikan dan berfungsi sesuai dengan tujuan pembelajaran. Program grafik 2D mampu menampilkan grafik fungsi satu variabel berdasarkan input fungsi dan rentang nilai sumbu X, sehingga mahasiswa dapat mengamati bentuk kurva dan perubahan nilai fungsi secara visual.

Selain itu, program grafik 3D mampu memvisualisasikan fungsi dua variabel dalam ruang tiga dimensi dalam bentuk permukaan grafik. Visualisasi ini memudahkan pemahaman hubungan antara variabel x , y , dan z , yang merupakan materi penting dalam Kalkulus II, khususnya pada pembahasan fungsi dua variabel dan grafik permukaan.

Hasil visualisasi yang dihasilkan dari kedua program tersebut membantu mahasiswa memahami konsep kalkulus yang bersifat abstrak menjadi lebih konkret dan mudah dipahami. Fitur interaktif seperti pengaturan fungsi, pengubahan rentang nilai, serta pengendalian tampilan grafik memberikan fleksibilitas bagi pengguna dalam mengeksplorasi karakteristik fungsi matematika. Dengan demikian, proyek ini dapat dijadikan sebagai media pendukung pembelajaran yang relevan dan aplikatif dalam menunjang pemahaman materi Kalkulus II.